Ассистент сервисной службы на Graph RAG

Автор: Екатерина Токарева.

Репозиторий: etokareva/llm-project.

1. Задача

Диспетчер сервисной службы получает модель оборудования и код ошибки. Для подготовки ремонта ему необходимо определить отказавший узел, подобрать деталь и её актуальную замену, проверить наличие и цену в ERP. Эти сведения распределены между руководствами, бюллетенями, регламентами и отчётами.

Цель проекта — автоматизировать поиск и сравнить Graph RAG с готовым плоским RAG на одинаковых вопросах. Заказ создаётся только после подтверждения диспетчером.

Пример ошибки baseline: на вопрос «Какой узел чаще всего отказывает в линейке «Арктика»?» система ответила «В базе знаний не найдено» вместо эталонного «пусковое реле». Для ответа требуется объединить сведения о моделях линейки и отказах; найденных отдельных фрагментов оказалось недостаточно.

2. Решение

Индекс строится заранее: документы разбиваются на фрагменты, модель извлекает сущности и связи, код собирает граф и сообщества, затем модель формирует сводки. При ответе используются граф, сводки и инструменты ERP.

Модуль	Реализация и назначение
`extraction.py`	JSON по заданной схеме; нормализация кодов и названий; проверка кодов по исходному фрагменту и названий по словарю. При некорректном ответе — одна повторная попытка, затем пустой результат
`graph_builder.py`	Направленный граф NetworkX. Сущности объединяются по типу и имени. Для узлов и связей сохраняются источники; вес связи равен числу уникальных фрагментов
`communities.py`	Лувен с фиксированным `seed`. Статистика вычисляется кодом по весам рёбер; модель формулирует сводку до 150 слов
`search.py`	Local отбирает факты вокруг сущностей вопроса; Global объединяет частичные ответы по сводкам через map/reduce
`tools.py`	Пять инструментов поиска и ERP с проверкой аргументов и обработкой ошибок
`agent.py`	Цикл вызовов инструментов, проверка происхождения фактов, повторы ERP и подтверждение заказа. Вспомогательные проверки вынесены в `validation.py`

Local: коды и названия сущностей находятся в вопросе программно, без обращения к модели. Для нескольких опор выбираются пути длиной до трёх рёбер без учёта направления, для одной — окрестность радиуса два. Для нескольких опор дополнительно включаются цепочки замен. Модель получает пронумерованные факты объёмом до 8000 символов. При неизвестном коде или отсутствии фактов ответ возвращается без вызова модели.

Global: для каждой сводки модель возвращает частичный ответ и оценку полезности. Некорректные результаты и оценки 0 пропускаются; остальные объединяются одним вызовом модели. Если полезных ответов нет, возвращается отказ.

Агент: использует `search_knowledge`, `summarize_topic`, `check_stock`, `get_price` и `create_service_order`. По умолчанию допускает до шести вызовов инструментов. Повтор одинакового вызова получает сохранённый результат. При недоступности ERP выполняются ещё две попытки; отсутствие данных, нулевой остаток и отклонённый запрос различаются.

До выполнения проверяются аргументы и происхождение номера детали. Перед выдачей ответа проверяются распознаваемые коды, цены и количества. При расхождении модель получает одну попытку исправления. Номер уже созданного заказа и сообщение о недоступности ERP сохраняются независимо от последующей генерации.

Разделение этапов облегчает проверку ошибок. Расчёты вынесены из модели в код, чтобы избежать неверного сравнения чисел. Несовместимые со схемой связи отбрасываются; встречные `REPLACED_BY` разрешаются по весу, при равенстве удаляются обе.

Проходят **215 тестов**, Ruff завершается без замечаний. Это подтверждает проверенные сценарии, но не гарантирует правильность всех ответов модели.

Проверка связки поиска и ERP

Дополнительный вопрос: «Найди в документах актуальную замену детали P-1042, затем узнай цену найденной детали в ERP».

Агент выполнил два последовательных шага:

1. search_knowledge нашёл связь P-1042 —REPLACED_BY→ P-1042B

   с источником bulletin_SB-2025-03.md.

2. get_price получил для P-1042B цену 2630 руб.

   и discontinued=False.

Итоговый ответ назвал актуальную замену и её цену. Пример проверен отдельно и не включён в метрики 24 вопросов.

3. Результаты

Условия эксперимента

- Генерация: Qwen3 4B Instruct 2507, Q4_K_M.
- Имя в API: `qwen3-4b-instruct-2507-q4_K_M`.
- Эмбеддинги: `bge-m3`, вектор 1024.
- Сервер: llama.cpp, build 12167, commit `240d68fcf`.
- Ускорение: AMD Radeon 880M, Vulkan.
- Контекст: 8192 токена; лимит извлечения — 4096 токенов.
- Открытый набор: 24 вопроса.
- Результаты эксперимента: `artifacts/`.

llama.cpp выбран для локального запуска. Предмет сравнения — четыре режима ассистента в одной конфигурации.

Качество ответов

Все значения в таблице — проценты правильных ответов по автоматической оценке. Размеры групп: локальные — 5, цепочки фактов — 7, обзорные — 4, инструменты — 5, отказы — 3.

Режим	Все вопросы, %	Локальные, %	Цепочки фактов, %	Обзорные, %	Инструменты, %	Отказы, %
Baseline	50,0	100,0	57,1	0,0	0,0	100,0
Local	29,2	40,0	0,0	50,0	0,0	100,0
Global	29,2	60,0	0,0	25,0	0,0	100,0
Agent	33,3	60,0	0,0	25,0	20,0	100,0

Baseline правильно ответил на **12 из 24** вопросов, Local и Global — на **7**, Agent — на **8**. Агент решил один из пяти вопросов с инструментами. Графовые режимы не решили вопросы на цепочки фактов.

Все режимы правильно обработали три вопроса на отказ. Автоматическая проверка не обнаружила выдуманных фактов, однако она не выявляет все ошибки рассуждения и неполные ответы.

Источники и вычислительные затраты

Режим	Полнота источников (Recall), %	Вызовов модели на вопрос, среднее	Токенов на вопрос, среднее	Ответов с обнаруженными выдуманными фактами, %
Baseline	85,6	1,00	1 345	0

Режим	Полнота источников (Recall), %	Вызовов модели на вопрос, среднее	Токенов на вопрос, среднее	Ответов с обнаруженными выдуманными фактами, %
Local	53,7	0,71	304	0
Global	39,7	17,42	7 421	0
Agent	44,4	5,42	3 509	0

Recall источников показывает долю найденных эталонных групп документов: достаточно указать хотя бы один документ группы. Показатель усредняется по вопросам с эталонными источниками.

Вызовы и токены усреднены по 24 вопросам. Токены включают запрос и ответ; значения характеризуют объём обработки, а не время или денежную стоимость. Кеш снижает фактические затраты повторных запусков.

Baseline лидирует по общей точности и полноте источников. Local использует меньше вызовов и токенов, в том числе из-за ранних отказов. Global требует больше всего обращений при той же общей точности, что Local. Доля ошибочных или отклонённых вызовов инструментов агента — **8,7%**.

Извлечение и индексация

Извлечение оценено на 16 документах с эталонной разметкой.

Объекты	Точность (Precision), %	Полнота (Recall), %
Сущности	100,0	92,1
Связи	81,9	50,9

Precision — доля правильных элементов среди извлечённых; Recall — доля найденных эталонных элементов. На оценочном наборе отброшено 12 элементов. Счётчик `dropped` учитывает сущности и связи совместно.

Полный индекс содержит **92 фрагмента, 104 узла, 103 связи и 17 сообществ**. Отброшено 104 элемента; конфликтов встречных замен — 0.

Индексация учитывает 109 обращений, 51 552 входных и 31 610 выходных токенов. При последнем повторном запуске все 109 обращений обслужены из кеша: токены отражают сохранённые данные, а не повторную генерацию.

Главное ограничение — полнота связей: наличие сущностей не восстанавливает пропущенные пути между ними.

Воспроизводимость

На отдельном клоне после настройки .env для llama.cpp успешно выполнены сборка контейнеров, индексация, тесты, Ruff и оценка всех режимов. Сохранённый кеш использован, итоговые метрики воспроизвелись.

4. Где граф помогает и где уступает

Выигрыш: обзорный вопрос

Вопрос: «Какой узел чаще всего отказывает в линейке «Арктика»?»

Baseline: «В базе знаний не найдено».

Global: «Пусковое реле чаще всего отказывает в линейке «Арктика»».

Ответ Global соответствует эталону. Обобщение по сводкам позволило ответить там, где поиск отдельных фрагментов не справился. Однако правильный итог не доказывает полноту статистики по линейке: сводки содержат только внутренние связи сообществ.

Проигрыш: конкретный регламент

Вопрос: «По какому регламенту устраняют ошибку E-17?»

Baseline: «По регламенту SP-104».

Local и Global: «В базе знаний не найдено».

В итоговом графе отсутствует связь `ErrorCode:E-17 —FIXED_BY→ Procedure:SP-104`, хотя факт есть в документах. Для Local нужный путь недоступен. Связь могла быть пропущена моделью или отброшена проверками; без исходного ответа извлечения эти причины не различаются.

Ограничение автоматической оценки

В предыдущем прогоне Local получил 75% на обзорных вопросах. Ручная проверка выявила два ложноположительных результата:

- **dev-13:** модель подменила компоненты моделями оборудования, считала связи вместо их весов и оборвала рассуждение без корректного вывода.
- **dev-15:** ответ закончился отказом, но поле `refused` осталось `False`; упоминание эталонной модели привело к положительной оценке.

После исправления итогового отказа dev-15 получил `refused=True`. Оценка обзорных ответов снизилась до 50%, общая точность Local — до 29,2%. Ложноположительный dev-13 сохраняется.

В dev-14 итог «циркуляционный насос» соответствует эталону, но подсчёт требует проверки по весам. В dev-16 получен отказ. Поэтому автоматические 50% не означают два содержательно правильных обзорных ответа.

5. Ошибки и эксперименты

Ошибка и этап	Причина и последствия	Изменение или ограничение
Неверные связи — извлечение	В `manual_AR-210.md#0` получены связи `Part –HAS_PART→ Component` вместо допустимых `Model –HAS_PART→ Part` и `Part –IS_A→ Component`. Проверки удалили их, разорвав путь к детали	В итоговом графе не устранено; контрольный вопрос E-17 на AR-210 завершился отказом
Потеря контекста — сводки	Связи между сообществами не входят в контекст. Модели, линейки и детали могут оказаться разделены	Статистика сообщества не объявляется полной статистикой линейки; требуется отдельный эксперимент
Неверное сравнение чисел — генерация сводки	В предварительном прогоне датчик с весом 3 назван более частым отказом, чем реле с весом 5	Код рассчитывает ранжирование и максимум, модель только формулирует результат
Ложноположительный ответ — оценка	Эталонный код или название встречается в ошибочном рассуждении, поэтому ответ засчитывается	Автоматическая оценка дополнена ручным разбором; итоговый отказ распознаётся отдельно

При разработке также проверялась расширенная инструкция извлечения. В предварительном эксперименте с Ollama Precision связей вырос с 82% до 100%, но точность агента снизилась с 9/24 до 7/24. Изменение отменено: для ответа важна полнота цепочки, а не только точность отдельных связей. Эти числа не относятся к итоговой конфигурации из раздела 3.

6. Границы системы и подтверждение

Создание заказа разрешено только при `confirm=True`. Иначе агент возвращает `needs_confirmation=True` и параметры `pending_action`, не вызывая инструмент создания заказа. Количество должно быть явно задано пользователем; при отсутствии значения запрашивается уточнение.

Проверен вопрос: «Оформи заказ: 2 штуки P-1042B для AR-210, склад Казань».

Без `--confirm` получен запрос подтверждения с параметрами:

```
{
```

```
  "tool": "create_service_order",
  "args": {
    "part_id": "P-1042B",
    "warehouse": "Казань",
    "qty": 2,
    "model": "AR-210"
  }
}
```

При повторном запуске с `--confirm` учебная ERP создала заказ **SO-000001** с теми же параметрами и статусом `created`. Номер вошёл в ответ. Поиск по документам не выполнялся: параметры переданы пользователем.

CLI запускает обработку заново, а не продолжает сохранённое действие. Поэтому текущий интерфейс не гарантирует неизменность параметров между запросом подтверждения и повторным запуском.

Проверка кодов и чисел ограничена поддерживаемыми текстовыми формами. Она не является полной смысловой проверкой. Отсутствие ответа в графе также не доказывает отсутствия факта в исходных документах.

7. Следующий шаг

Идея: включить в контекст сводки связи, пересекающие границу сообщества, с указанием источников.

Гипотеза: это уменьшит потерю контекста между моделями, линейками и деталями и улучшит обзорные ответы.

Проверка: сравнить два варианта на одном графе, с одной моделью и теми же 24 вопросами. Измерить точность, вызовы и токены; вручную проверить четыре обзорных ответа. Пропущенные при извлечении связи такое изменение не восстановит.

8. Вклад автора

Проект выполнен индивидуально Екатериной Токаревой: реализованы шесть модулей, подключены инструменты учебной ERP, настроен llama.cpp, проведены тестирование,

рефакторинг, оценка четырёх режимов и разбор ошибок. Подготовлены результаты экспериментов и отчёт.

9. Использование ИИ-инструментов

Codex использовался для объяснения требований, подготовки и исправления кода, рефакторинга, разработки дополнительных тестов, анализа результатов и редактирования отчёта.

Предложенные изменения применялись и проверялись запуском тестов, Ruff и команд проекта. Числовые результаты получены из фактических прогонов.